



Prueba Técnica: LLM Engineer

 SG TECH

 MICROSERVICIO PYTHON

SG Tech solicita la construcción de un microservicio técnico que simule una versión reducida de un motor de IA empresarial orientado al análisis de datos estructurados y a la generación asistida de hallazgos y borradores técnicos.

Contexto y Capacidades Requeridas

La solución debe demostrar capacidad para los siguientes aspectos clave del procesamiento de datos e inteligencia artificial aplicada:



Procesar datos estructurados



Ejecutar cálculos deterministas simples



Recuperar contexto documental



Usar LLM solo para síntesis y redacción controlada



Devolver resultados estructurados y trazables



Aplicar buenas prácticas de desarrollo, seguridad y diseño

Objetivo y Salida Esperada



Construir un servicio en Python que reciba un dataset y un conjunto de documentos de contexto, procese la información y genere una salida estructurada con:

Entradas del servicio

- Dataset (CSV o JSON con datos estructurados)
- Metadata del dataset (periodo, unidades, segmentación)
- Documentos de contexto (TXT, PDF o Markdown)
- Prompt de negocio

Salida estructurada con

- Resultados analíticos básicos
- Hallazgos explicables
- Recomendaciones preliminares en estado **draft**
- Advertencias cuando falte contexto o existan inconsistencias

Insumos que recibirá el candidato

SG Tech entregará un paquete con estos archivos:

1

Dataset

Archivo CSV o JSON con datos estructurados de ejemplo.

2

Metadata

Archivo JSON con periodo, unidades, segmentación y atributos del dataset.

3

Documentos de contexto

3 a 5 archivos en TXT, PDF o Markdown con reglas, criterios técnicos y definiciones de referencia.

4

Esquema esperado

Un JSON Schema con la estructura mínima de salida.

5

Casos de prueba base

2 o 3 escenarios esperados de ejecución.

Alcance Esperado de la Solución

El candidato deberá construir un microservicio o aplicación ejecutable que permita:



Recepción de solicitud

Recibir una petición con referencia al dataset, metadata y prompt de negocio.



Procesamiento determinista

Calcular métricas simples en código, sin delegar al LLM operaciones críticas.



Recuperación de contexto

Seleccionar fragmentos relevantes de los documentos entregados para sustentar hallazgos.



Generación controlada

Usar LLM solo para redactar insights y borradores basados en datos calculados y contexto recuperado.

Salida estructurada

Devolver un JSON válido conforme al esquema entregado.

Trazabilidad

Registrar al menos dataset usado, documentos consultados, prompt base y versión/configuración usada.

Manejo de errores

Bloquear o degradar de forma segura cuando falten datos, contexto o consistencia mínima.

Requisitos Funcionales Obligatorios

API o ejecución técnica

La solución debe exponerse como una API con **FastAPI** o como una aplicación ejecutable claramente estructurada. Si se usa API, se recomienda al menos:

POST /jobs

Para iniciar procesamiento



GET /jobs/{job_id}

Para consultar estado y resultado



Se valorará positivamente simular procesamiento asíncrono, aunque no se exige una cola real.

Cálculo determinista

El candidato debe implementar en código al menos **3 cálculos simples**, por ejemplo:

- Promedio por periodo
- Desviación porcentual entre segmentos
- Acumulado por categoría
- Detección básica de anomalía por regla definida

RAG o recuperación de contexto

El candidato debe implementar una estrategia de recuperación documental suficiente para demostrar grounding. No se exige vector DB obligatoria; puede resolverse con enfoques simples pero defendibles.

LLM limitado a lenguaje

El LLM **no debe usarse** para calcular, interpretar reglas sin respaldo o producir resultados libres sin contexto. Solo debe completar estructuras predefinidas y redactar sobre evidencia recuperada.

Esquema de Salida Estructurada



La respuesta debe cumplir un esquema de salida fijo. A continuación se muestra la estructura mínima requerida:

```
{
  "job_id": "string",
  "status": "success|warning|error",
  "summary": {
    "period": "string",
    "dataset_ref": "string"
  },
  "metrics": [
    {
      "name": "string",
      "value": 0,
      "unit": "string",
      "source": "deterministic"
    }
  ],
  "findings": [
    {
      "id": "string",
      "type": "string",
      "statement": "string",
      "evidence": {
        "dataset_fields": ["string"],
        "document_refs": ["string"]
      },
      "confidence": "high|medium|low"
    }
  ],
  "recommendations_draft": [
    {
      "id": "string",
      "text": "string",
      "based_on_findings": ["string"],
      "disclaimer": "Draft subject to human review"
    }
  ],
  "warnings": [
    {
      "code": "string",
      "message": "string"
    }
  ],
  "trace": {
    "prompt_version": "string",
    "documents_used": ["string"],
    "rules_applied": ["string"]
  }
}
```

Requisitos de Ingeniería y Seguridad Obligatorios

La prueba **no debe resolverse como demo rápida ni como notebook caótico**. Aquí está la parte importante:

Estructura

El código debe evidenciar separación clara entre entrada/validación, lógica determinista, recuperación de contexto, invocación del LLM y serialización de salida.

Validación

Se deben validar inputs y outputs. Se recomienda uso de **Pydantic**, dataclasses o equivalente.

Secretos

No se permiten tokens, claves o endpoints hardcodeados. Debe incluirse archivo `.env.example`.

Errores

La solución debe manejar errores de forma controlada. No debe exponer stack trace crudo en respuestas funcionales.

Logs

Debe existir logging básico. No deben registrarse secretos ni payloads sensibles completos.

Archivos

Si el candidato procesa documentos locales, debe hacerlo de forma segura y controlada. No se valorará código que acepte rutas arbitrarias sin validación.

Patrones y diseño

Se valorará desacople técnico real. No se exige nombrar formalmente los patrones, pero sí demostrar criterio. Adapter, service layer, strategy, DTOs o separación por módulos suman valor.

Pruebas

Debe incluir al menos pruebas mínimas sobre validación, cálculo o contrato de salida.

Linux

La solución debe poder ejecutarse en entorno Linux o contenedor. README e instrucciones deben reflejarlo.

Restricciones y Entregables

Restricciones

- **Tiempo objetivo:** 6 a 8 horas efectivas
- **Ventana de entrega:** 48 horas desde el envío
- **Tecnología preferida:** Python
- **Framework sugerido:** FastAPI
- **Se permite:** uso de librerías de LLM, RAG, embeddings o recuperación documental
- **No se requiere:** frontend, despliegue cloud real, fine-tuning, infraestructura enterprise completa
- **No se permite:** entregar solo notebook desordenado como solución principal

Entregables

1. **Código fuente:** repositorio o archivo comprimido
2. **README:** instalación, ejecución, supuestos y estructura
3. **.env.example:** variables requeridas sin secretos reales
4. **Documento técnico breve:** máximo 2 páginas o sección ampliada en README explicando arquitectura, decisiones de seguridad, patrones aplicados y limitaciones
5. **Pruebas:** conjunto mínimo de tests
6. **Ejemplo de ejecución:** uno o más outputs generados
7. **Opcional:** Dockerfile

Criterios de Evaluación



La puntuación total es de **100 puntos**, distribuidos en los siguientes criterios:

20

Resolución funcional

Que la solución corra y cumpla el flujo principal.

20

Diseño y modularidad

Separación de responsabilidades, claridad estructural y mantenibilidad.

20

Seguridad aplicada

Validación, secretos, logs, errores, manejo básico seguro de archivos e integración.

15

Control de salida

Cumplimiento de esquema, trazabilidad, evidencia y bloqueo ante falta de contexto.

10

Calidad de código

Legibilidad, consistencia, tipado, orden y limpieza.

5

Pruebas

Cobertura mínima útil y criterio de validación.

10

Criterio técnico documentado

Capacidad para explicar decisiones, límites y riesgos.

Criterio

